

Sección A — Prueba Técnica

Contexto

Estamos construyendo el módulo de gestión de reservas de una plataforma de servicios. Queremos ver cómo abor das un problema con reglas de negocio reales, cómo estructuras tu solución y cómo trabajas con las herramientas que usas habitualmente.

Puedes usar las herramientas que prefieras, incluyendo asistentes de IA. No nos interesa si las usaste o no, nos interesa que entiendas y puedas defender lo que entregues.

Reglas y entregables

- Stack: libre. Usa el lenguaje, framework y librerías con los que te sientas más cómodo.
- Tiempo estimado: entre 4 y 6 horas. No queremos que dediques más; si algo no alcanza, documéntalo en el README.
- Entrega: repositorio en Git (GitHub, GitLab o similar) con el código, instrucciones para correrlo, y un README con tus decisiones.
- Plazo: 5 días calendario desde el envío de esta prueba.

El problema

Debes implementar un servicio que gestione la creación y cancelación de reservas para un sistema de citas. El servicio puede ser una API HTTP, una CLI o un módulo con funciones expuestas — tú decides la forma, pero debe ser ejecutable y testeable.

Funcionalidades requeridas

1. Crear una reserva: dado un usuario, un servicio y una fecha/hora, registrar la reserva si cumple las reglas.
2. Cancelar una reserva: dado un ID de reserva, marcarla como cancelada si las reglas lo permiten.
3. Listar reservas de un usuario en un rango de fechas.
4. Calcular el monto a cobrar o reembolsar al cancelar (ver reglas).

Reglas de negocio (léelas con cuidado)

Estas reglas son específicas y debes implementarlas todas. Si alguna te parece ambigua, toma una decisión razonable y documéntala en el README.

- **Horarios de operación:** se aceptan reservas de lunes a sábado entre 7:00 y 19:00 hora local de Bogotá (America/Bogota). Los domingos no se aceptan reservas. Los festivos de Colombia tampoco (puedes usar una lista hardcodeada de los festivos de 2026).
- **Anticipación mínima:** una reserva debe crearse al menos con 2 horas de anticipación respecto a su fecha de inicio.

- **Duración:** cada servicio tiene una duración fija (incluida en el catálogo, ver más abajo). Dos reservas del mismo profesional no pueden solaparse.
- **Cancelación estándar:** si el usuario cancela con más de 24 horas de anticipación, se reembolsa el 100%.
- **Cancelación tardía:** si cancela entre 24 y 4 horas antes, se reembolsa el 50%.
- **Cancelación muy tardía:** si cancela con menos de 4 horas, no hay reembolso.
- **Excepción usuarios premium:** los usuarios con plan premium tienen reembolso del 100% hasta 4 horas antes, y 50% entre 4 horas y 1 hora antes. Después de eso, no hay reembolso.
- **Excepción servicios no reembolsables:** los servicios marcados como `non_refundable` nunca tienen reembolso, sin importar el tiempo de anticipación ni el plan del usuario. Pero sí se pueden cancelar.
- **Límite por usuario:** un usuario no puede tener más de 3 reservas activas (no canceladas y futuras) al mismo tiempo.

Datos de ejemplo

Se incluye un archivo `data/seed.json` con usuarios, servicios y reservas de prueba (lo encontrarás adjunto al envío de esta prueba; si no, créalo tú con datos representativos y documenta la estructura). Los datos contienen algunas inconsistencias intencionales (ej. fechas en distintos formatos, campos faltantes) — decide cómo manejarlas.

Entregables específicos

5. Código funcional ejecutable, con instrucciones claras de instalación y ejecución.
6. Al menos 5 casos de prueba automatizados que cubran los escenarios que consideres más críticos.
7. Un `README.md` con: cómo correrlo, decisiones técnicas (qué elegiste y por qué), supuestos que tomaste, qué dejaste por fuera y por qué, y qué harías diferente con más tiempo.
8. Un archivo `NOTAS.md` (o sección dentro del `README`) donde describas: qué partes hiciste con ayuda de IA, qué partes ajustaste o reescribiste, y por qué. No hay respuesta correcta aquí, queremos transparencia.

Criterios de evaluación

- Correctitud: ¿funciona y cubre las reglas?
- Manejo de casos borde: ¿qué pasa con fechas raras, datos faltantes, concurrencia básica?
- Claridad del código: ¿se entiende?, ¿está bien organizado?
- Decisiones documentadas: ¿podemos entender por qué hiciste lo que hiciste?
- Calidad de las pruebas: ¿qué eligió probar el candidato y cómo?

Siguientes pasos

Tras la entrega tendremos una entrevista técnica de aproximadamente 1 hora donde te pediremos:

- Caminar por tu solución y explicar decisiones clave.

- Hacer una modificación pequeña en vivo (sobre el mismo código).
- Discutir trade-offs y posibles mejoras.

¡Éxitos! Cualquier duda durante el desarrollo, escríbenos.